

Examination Portal

Complete Application Flow Documentation

This document explains the end-to-end flow of the Examination Portal — from registration and login through every feature — for both Admin and Student roles. Each section covers the UI component, the API call made, and how data is stored in MySQL.

Section	Content
Scenario 1	Admin: Register → Login → Dashboard → Subjects → Questions → Tests → Users → Reports → Profile
Scenario 2	Student: Register → Login → Dashboard → Tests → Exam → Submit → Report → Performance → Profile
Appendix	Architecture overview, JWT flow, full DB schema summary

Architecture Overview

Tech Stack

Layer	Technology	Purpose
Frontend	React + Vite + Tailwind CSS	UI components, routing, API calls
Backend	Spring Boot (Java)	REST API, business logic, JWT auth
ORM	Hibernate / Spring Data JPA	Maps Java entities to MySQL tables
Database	MySQL 8	Persistent data storage
Auth	JWT (JSON Web Token)	Stateless authentication
Email	Spring Mail + Gmail SMTP	Send exam result emails to students

Request Flow

1. User interacts with a React UI component
2. Component makes an HTTP request with JWT token in the Authorization header
3. Spring Boot → JwtRequestFilter validates the token
4. SecurityConfig enforces role (ADMIN / STUDENT) → routes to the correct Controller
5. Controller → Service → Repository (Hibernate) → MySQL
6. Result travels back: MySQL → Repository → Service → Controller → JSON → React UI

JWT Authentication

After a successful login the backend generates a JWT token containing the username and role. The frontend stores it and sends it with every subsequent request:

```
Authorization: Bearer <token>
```

JwtRequestFilter intercepts every request, validates the token, and sets the security context. Protected endpoints use @PreAuthorize to enforce ADMIN or STUDENT access.

SCENARIO 1: Admin Flow

Covers everything an admin does: registration, login, and all management pages.

Step 1 — Register as Admin

UI: RegisterComponent.jsx | Route: /register

The admin opens /register, fills in username, email, password, selects role = ADMIN, and optionally first/last name.

API Call:

Method	Endpoint	Description
POST	/api/auth/register	Create a new user account

Request payload (RegisterRequest DTO):

```
{ "username": "admin1", "email": "admin@exam.com", "password": "secret123", "role": "ADMIN", "firstName": "John", "lastName": "Doe" }
```

Backend steps:

- AuthController calls AuthService.register()
- Checks username and email uniqueness in users table
- BCrypt-encodes the password
- Saves User entity → Hibernate → INSERT into users
- Generates JWT and returns AuthResponse

DB — users table (row created):

Column	Example Value	Notes
id	1 (auto)	AUTO_INCREMENT
username	admin1	UNIQUE
email	admin@exam.com	UNIQUE
password	\$2a\$10\$...	BCrypt hashed
role	ADMIN	CHECK: ADMIN or STUDENT
first_name	John	
last_name	Doe	
is_active	true	DEFAULT TRUE
created_at	2026-06-01 10:00:00	AUTO

Step 2 — Login as Admin

UI: LoginComponent.jsx | Route: /login

Admin enters username (or email) and password. On success, redirected to /admin/dashboard.

API Call:

Method	Endpoint	Description
POST	/api/auth/login	Authenticate and receive JWT token

Request payload (LoginRequest DTO):

```
{ "username": "admin1", "password": "secret123" }
```

Response (AuthResponse DTO):

```
{ "token": "eyJhbGc...", "type": "Bearer", "id": 1, "username": "admin1", "email": "admin@exam.com", "role": "ADMIN" }
```

Backend steps:

- Looks up user by username OR email (UserRepository)
- AuthenticationManager verifies password against BCrypt hash
- JwtUtil.generateToken() creates signed JWT with username + role
- Frontend stores token in localStorage via AuthContext

No DB write on login — only a SELECT on the users table.

Step 3 — Admin Dashboard

UI: AdminDashboardComponent.jsx | **Route:** /admin/dashboard

Displays stat cards: Total Users, Total Students, Subjects, Questions, Tests. Quick-action links to Manage Users, Manage Tests, View Reports.

API Calls on load:

Method	Endpoint	Description
GET	/api/admin/users	Fetch all users → derive counts
GET	/api/admin/subjects	Fetch subjects → count
GET	/api/admin/questions	Fetch questions → count
GET	/api/admin/tests	Fetch tests → count

Read-only page — no DB writes.

Step 4 — Subject Management

UI: SubjectManagementComponent.jsx | **Route:** /admin/subjects

View all subjects in a table. A toggle form above the table handles add and edit. Deleting a subject cascades to its questions and tests.

API Calls:

Method	Endpoint	Description
GET	/api/admin/subjects	Load all subjects on page open
POST	/api/admin/subjects	Create a new subject
PUT	/api/admin/subjects/{id}	Update an existing subject
DELETE	/api/admin/subjects/{id}	Delete subject (cascades to questions, tests)

DB — subjects table (row created on POST):

Column	Example Value	Notes
id	1 (auto)	AUTO_INCREMENT
name	Java Programming	UNIQUE, required
description	Core Java concepts	TEXT, optional
is_active	true	DEFAULT TRUE
created_at	2026-06-01 10:05:00	AUTO

Step 5 — Question Management

UI: QuestionManagementComponent.jsx | Route: /admin/questions

Create MCQ questions linked to a subject: 4 answer options, correct option (1–4), marks per question, and difficulty (EASY / MEDIUM / HARD). Questions can be edited or deleted.

API Calls:

Method	Endpoint	Description
GET	/api/admin/questions	Load all questions
GET	/api/admin/subjects	Populate subject dropdown
GET	/api/admin/questions/subject/{id}	Filter questions by subject
POST	/api/admin/questions	Create a new question
PUT	/api/admin/questions/{id}	Edit a question
DELETE	/api/admin/questions/{id}	Delete a question

DB — questions table (row created on POST):

Column	Example Value	Notes
id	1 (auto)	
subject_id	1	FK → subjects.id
question_text	What is int size in Java?	TEXT, required
option1	8 bits	VARCHAR(500)
option2	16 bits	VARCHAR(500)
option3	32 bits	VARCHAR(500)
option4	64 bits	VARCHAR(500)
correct_option	3	CHECK: 1–4
marks	1	INT, default 1
difficulty	EASY	EASY / MEDIUM / HARD
is_active	true	

Step 6 — Test Management

UI: TestManagementComponent.jsx | Route: /admin/tests

Create a test with title, subject, duration, total marks, and passing marks. After creation a question selector panel appears — only questions for the selected subject are shown. Admin checks questions to attach them to the test.

API Calls:

Method	Endpoint	Description
GET	/api/admin/tests	Load all tests
GET	/api/admin/subjects	Populate subject dropdown
GET	/api/admin/questions	Load questions for selector
POST	/api/admin/tests	Create a new test
POST	/api/admin/tests/{testId}/questions	Add multiple questions to test
DELETE	/api/admin/tests/{testId}/questions/{questionId}	Remove a question from test
DELETE	/api/admin/tests/{id}	Delete a test

DB — tests table (row created on POST):

Column	Example Value	Notes
id	1 (auto)	
title	Java Basics	
subject_id	1	FK → subjects.id
duration_minutes	30	INT
total_marks	10	INT
passing_marks	6	INT
created_by	1	FK → users.id (admin)
is_active	true	

DB — test_questions join table (one row per question added):

Column	Example Value	Notes
id	1 (auto)	
test_id	1	FK → tests.id
question_id	1	FK → questions.id
question_order	1	Display order

Marks per question = total_marks ÷ number_of_questions, calculated dynamically in the UI.

Step 7 — User Management

UI: `UserManagementComponent.jsx` | Route: `/admin/users`

View all registered users with filter buttons (ALL / ADMIN / STUDENT). Each row shows ID, username, email, role badge, active status, and a Delete button.

API Calls:

Method	Endpoint	Description
GET	<code>/api/admin/users</code>	Fetch all users
GET	<code>/api/admin/users/role/{role}</code>	Filter by ADMIN or STUDENT
DELETE	<code>/api/admin/users/{id}</code>	Delete a user

ON DELETE CASCADE: deleting a user also removes their `exam_attempts` and `student_answers`.

Step 8 — Reports

UI: `AdminReportsComponent.jsx` | Route: `/admin/reports`

Two tabs: Student-wise Report (attempts, avg score, pass rate per student) and Subject-wise Report (attempts, avg score, pass rate per subject).

API Calls:

Method	Endpoint	Description
GET	<code>/api/admin/reports/students</code>	Aggregated student performance
GET	<code>/api/admin/reports/subjects</code>	Aggregated subject performance

ReportService runs SELECT + aggregate queries on `exam_attempts` joined with users and subjects. Read-only — no DB writes.

Step 9 — Admin Profile

UI: `AdminProfileComponent.jsx` | Route: `/admin/profile`

Displays the logged-in admin's username, email, name, role badge, account status, and created/updated timestamps.

API Call:

Method	Endpoint	Description
GET	<code>/api/admin/profile</code>	Backend reads username from JWT, fetches the User record from DB

SCENARIO 2: Student Flow

Covers everything a student does: registration, login, taking exams, reviewing results, checking performance analytics, and viewing their profile.

Step 1 — Register as Student

UI: RegisterComponent.jsx | Route: /register

Same form as admin registration. Student selects role = STUDENT.

API Call:

Method	Endpoint	Description
POST	/api/auth/register	Register a new student account

DB — users table (row created, role = STUDENT):

Column	Example Value	Notes
id	2 (auto)	
username	student1	UNIQUE
email	student@test.com	UNIQUE
password	\$2a\$10\$...	BCrypt hashed
role	STUDENT	
first_name	Jane	
last_name	Smith	
is_active	true	

Step 2 — Login as Student

UI: LoginComponent.jsx | Route: /login

Same login form. On success the JWT is stored and the student is redirected to /student/dashboard.

API Call:

Method	Endpoint	Description
POST	/api/auth/login	Authenticate and receive JWT token

Step 3 — Student Dashboard

UI: StudentDashboardComponent.jsx | Route: /student/dashboard

Shows 6 stat cards (Total Attempts, Avg Score, Highest Score, Lowest Score, Tests Passed, Tests Failed), quick-action links, and a Recent Attempts list showing the last 5 results.

API Calls on load:

Method	Endpoint	Description
GET	/api/student/performance	Fetch stats: totals, avg/high/low scores, pass/fail counts, recent attempts
GET	/api/student/tests	Fetch active tests (for 'X tests available' count in quick actions)

Read-only — backend queries exam_attempts for the current student.

Step 4 — Available Tests

UI: AvailableTestsComponent.jsx | Route: /student/tests

All active tests shown as cards with title, subject, duration, total marks, passing marks, and question count. Clicking 'Start Test' creates an exam attempt and navigates to the exam page.

API Calls:

Method	Endpoint	Description
GET	/api/student/tests	Fetch all active tests
POST	/api/student/exams/start/{testId}	Create exam attempt, redirect to exam

DB — exam_attempts table (row created on Start Test):

Column	Value on Creation	Notes
id	1 (auto)	
student_id	2	FK → users.id
test_id	1	FK → tests.id
start_time	2026-06-01 11:00:00	AUTO
end_time	NULL	Filled on submission
score	0	Updated after submit
status	IN_PROGRESS	
total_questions	0	Updated after submit
correct_answers	0	Updated after submit
wrong_answers	0	Updated after submit
unanswered	0	Updated after submit

Step 5 — Taking the Exam

UI: ExamPageComponent.jsx | Route: /student/exam/{testId}

Shows title, subject, countdown timer, one question at a time with 4 clickable option buttons. A question navigation panel at the bottom shows all question numbers (grey = unanswered, green = answered, blue = current). Student can navigate freely.

API Calls during exam:

Method	Endpoint	Description
GET	/api/student/exams/{attemptId}/questions	Load all questions for this attempt (correct answers are hidden from response)
POST	/api/student/exams/answer?attemptId={id}	Save a single answer each time student selects an option

Save Answer payload (AnswerRequest DTO):

```
{ "questionId": 1, "selectedOption": 3 }
```

DB — student_answers table (one row per answered question):

Column	Example Value	Notes
id	1 (auto)	
exam_attempt_id	1	FK → exam_attempts.id
question_id	1	FK → questions.id
selected_option	3	1–4 (NULL if skipped)
is_correct	true	Compared to correct_option
answered_at	2026-06-01 11:05:00	AUTO

UNIQUE constraint on (exam_attempt_id, question_id) — re-selecting an option updates the existing row rather than inserting a duplicate.

Step 6 — Submitting the Exam

UI: ExamPageComponent.jsx — 'Submit Exam' button on last question, or auto-submit when timer hits 0.

API Call:

Method	Endpoint	Description
POST	/api/student/exams/submit	Save remaining answers and finalize the attempt

Request payload (SubmitExamRequest DTO):

```
{ "attemptId": 1, "answers": [ {"questionId": 1, "selectedOption": 3}, ... ] }
```

ExamService.submitExam() logic:

- Saves any answers included in the request
- Fetches all student_answers for this attempt
- Compares each selectedOption with the question's correctOption
- Counts: correct, wrong, unanswered (total - answered)
- Calculates score = (correctAnswers / totalQuestions) × totalMarks
- UPDATES exam_attempts: score, counts, end_time, status = COMPLETED

DB — exam_attempts row after submission:

Column	Updated Value	Notes
end_time	2026-06-01 11:28:00	Time of submission
score	7.50	DECIMAL(5,2)

Column	Updated Value	Notes
total_questions	5	
correct_answers	3	
wrong_answers	1	
unanswered	1	
status	COMPLETED	

Step 7 — Test Report / Result Page

UI: TestReportComponent.jsx | **Route:** /student/report/{attemptId}

Displays PASSED / FAILED badge, score summary (correct, wrong, unanswered, passing marks), and a full question-by-question analysis with colour-coded answers: green = correct, red = wrong, grey = unanswered. Each question reveals the correct answer.

API Calls:

Method	Endpoint	Description
GET	/api/student/exams/attempts/{attemptId}	Fetch attempt details and all student answers
GET	/api/student/exams/{attemptId}/answers	Fetch answer list with full question data for analysis
POST	/api/student/email-results/{attemptId}	Send result email via Gmail SMTP to student's registered address

Step 8 — Performance Reports

UI: PerformanceReportsComponent.jsx | **Route:** /student/performance

Shows 4 stat cards, a Subject-wise bar chart (Avg / Min / Max score per subject using Recharts), a Recent Scores bar chart (last 10 attempts), and a full attempt history table with a 'View Report' link per row.

API Calls:

Method	Endpoint	Description
GET	/api/student/performance	Overall stats: total attempts, avg, high, low scores
GET	/api/student/performance/subjects	Per-subject breakdown: attemptCount, avgScore, minScore, maxScore
GET	/api/student/exams/attempts	Full list of completed attempts for history table and chart

All data from aggregating exam_attempts filtered by the student's ID. Read-only.

Step 9 — Student Profile

UI: ProfileComponent.jsx | **Route:** /student/profile

Displays avatar initial, full name, username, email, role badge, account status, and created/updated timestamps.

API Call:

Method	Endpoint	Description
GET	/api/student/profile	Backend reads username from JWT and returns the User record

Appendix: Database Schema Summary

All 7 tables, their key columns, and which parts of the application write to or read from them.

Table	Key Columns	Written By	Read By
users	id, username, email, password, role, first_name, last_name, is_active, created_at	Register (admin & student)	Login, Profile pages, Admin User Mgmt, Reports
subjects	id, name, description, is_active, created_at	Admin: Subject Management (create/update)	Admin Dashboard, Question Mgmt, Test Mgmt, Student Tests page
questions	id, subject_id, question_text, option1–4, correct_option, marks, difficulty	Admin: Question Management (create/update)	Admin Question Mgmt, Test question selector, Exam Page, Test Report
tests	id, title, subject_id, duration_minutes, total_marks, passing_marks, created_by, is_active	Admin: Test Management (create)	Admin Test Mgmt, Student Available Tests, Exam Page
test_questions	id, test_id, question_id, question_order	Admin: Add questions to test	Exam Page (load question set), Test detail view
exam_attempts	id, student_id, test_id, start_time, end_time, score, correct/wrong/unanswered, status	Student: Start Exam (INSERT), Submit Exam (UPDATE)	Student Dashboard, Performance, Test Report, Admin Reports
student_answers	id, exam_attempt_id, question_id, selected_option, is_correct, answered_at	Student: Save Answer during exam, Submit Exam	Test Report (question-by-question analysis)

Foreign Key Relationships

```
users (1) ■■→ (many) tests [tests.created_by → users.id]
users (1) ■■→ (many) exam_attempts [exam_attempts.student_id → users.id]
subjects (1) ■■→ (many) questions [questions.subject_id → subjects.id]
subjects (1) ■■→ (many) tests [tests.subject_id → subjects.id]
tests (many) ←■■→ (many) questions [via test_questions join table]
exam_attempts (1) ■■→ (many) student_answers [student_answers.exam_attempt_id]
questions (1) ■■→ (many) student_answers [student_answers.question_id]
```

All foreign keys use ON DELETE CASCADE — deleting a parent record automatically removes all related child records (e.g., deleting a subject removes its questions, tests, and all associated exam attempts and student answers).